

Blastline AI

AWS security auditing and remediation

Technical Documentation

Ritvik Indupuri

1 Overview

Blastline AI is a web application for examining AWS security posture, investigating possible attack paths, and managing remediation. It combines API-based configuration checks with model-assisted interpretation, then keeps findings, proposed changes, execution evidence, and audit history in PostgreSQL.

An operator connects an AWS account, selects services, and starts an audit. The backend reads AWS metadata, evaluates configuration checks, and asks Gemini to critique findings, suggest attack chains, and draft fixes. Separate tools support change-impact exploration, principal activity review, and infrastructure-plan review.

Audits collect configuration and event metadata. Applying a remediation is a separate write operation: the server requires an approved remediation record, translates its snippet into AWS API actions, executes those actions, and attempts post-change verification.

Core capabilities

Area	Implemented behavior
Posture assessment	Selected AWS checks, findings, control references, severity and risk scoring.
Investigation	Interactive attack-path graphs, model-assisted impact and permission exploration, CloudTrail principal replay.
Change review	Terraform and CloudFormation plan interpretation with per-change assessments.
Remediation	Fix proposals, approval state, API execution, verification evidence and failure refinement.
Reporting	Audit transcripts, finding differences, history, CSV and summary PDF export.

Document guide

Sections 2–3 describe the architecture and audit flow. Sections 4–6 explain check coverage, models, and investigation tools. Sections 7–8 cover remediation. Sections 9–11 document persistence, interfaces, and setup. Section 12 provides implementation references and next steps.

This document describes the reviewed application code. Model-generated assessments are presented as analysis, while collected AWS fields and execution responses are treated as evidence. Live AWS deployment behavior was not exercised for this documentation.

2 System architecture

The browser uses Supabase for identity and application state. Deno Edge Functions coordinate AWS requests and model calls; PostgreSQL supplies the shared history and evidence store.

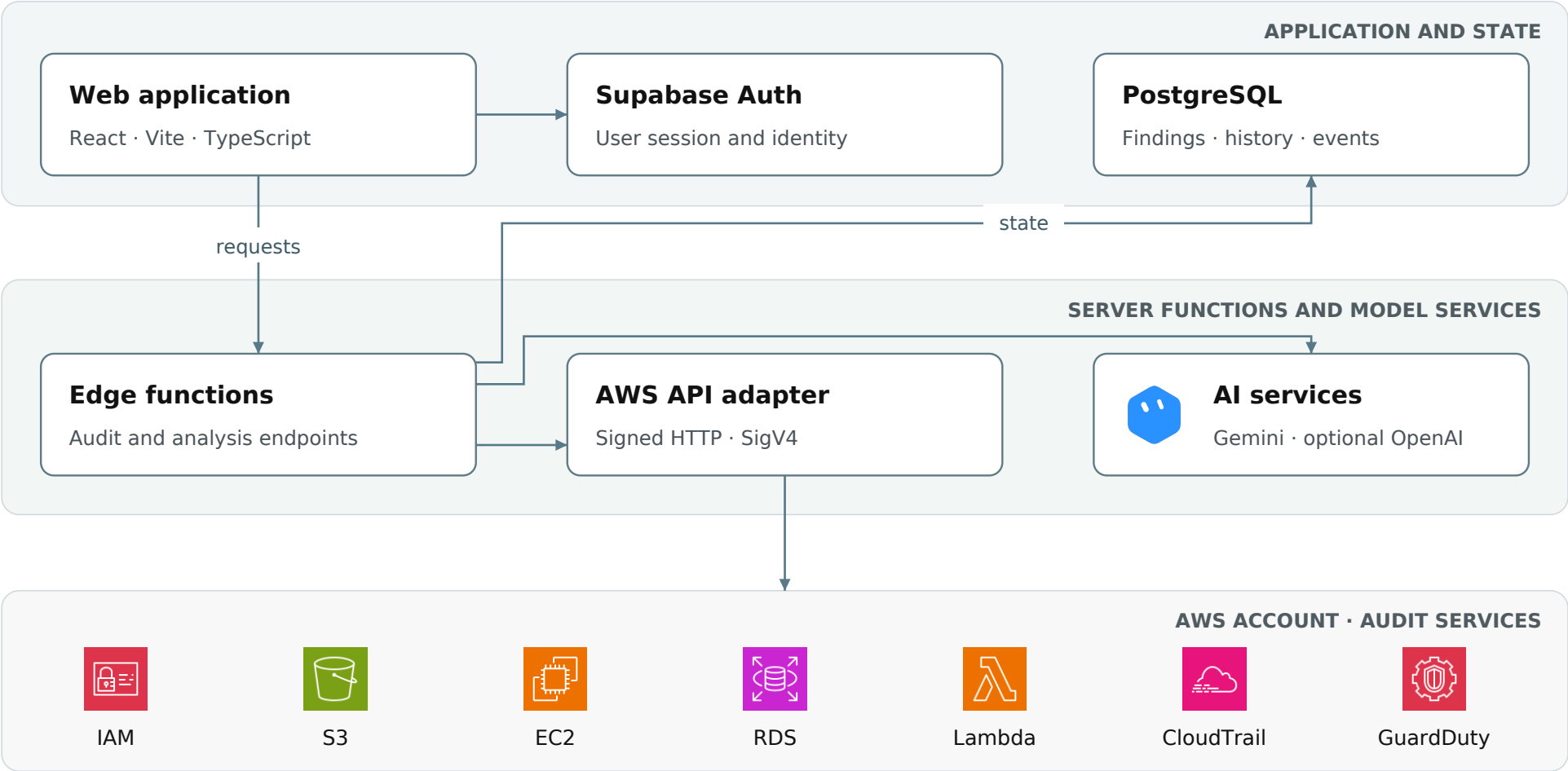


Figure 1. Application boundaries and the principal external services. Arrows show logical interactions.

The adapter uses AWS Signature Version 4. Gemini calls use the Lovable gateway; OpenAI is an optional direct route for failure analysis. The seven icons represent the current audit selector. Additional backend checks are described in Section 4.

3 Audit and agent lifecycle

The audit pipeline runs a fixed sequence of collection, analysis, and reporting stages. Agent icons identify the stages that make language-model calls.

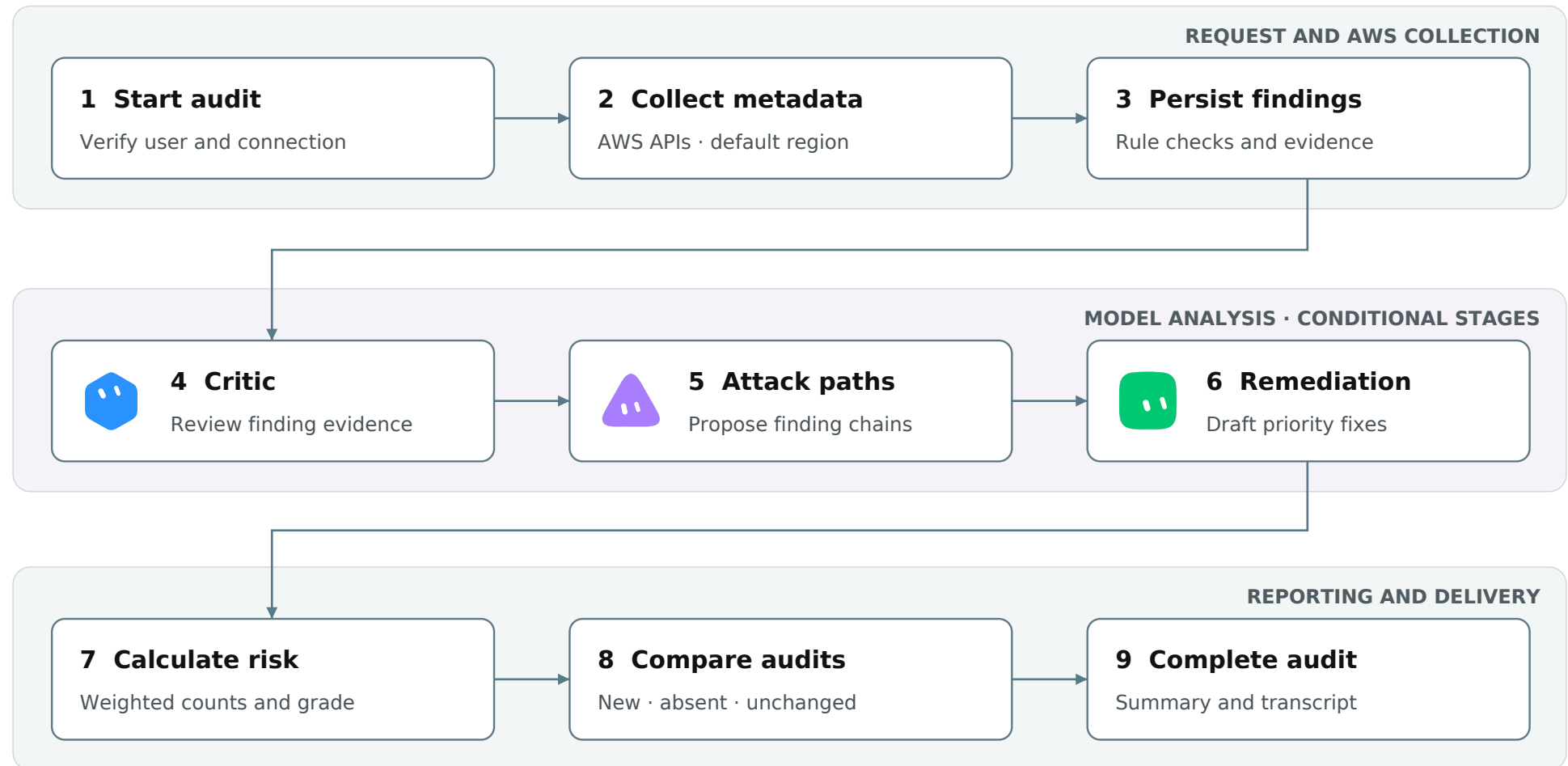


Figure 2. Audit execution order. Conditional model stages are skipped when their input criteria are not met.

Critic review requires at least one finding; attack-path synthesis requires two or more. Remediation drafting uses up to eight high or critical findings. These stages run sequentially, rather than as independent agents exchanging messages. The recon, misconfig, and reporter names in the transcript refer to procedural code stages.

4 AWS collection and security checks

The audit selector currently offers IAM, S3, EC2, RDS, Lambda, CloudTrail, and GuardDuty. The pipeline also contains branches for KMS, ECR, Secrets Manager, EKS, and VPC when those service identifiers are supplied in the audit scope.

Connection verification calls STS GetCallerIdentity using the stored access-key pair. Regional collectors use the connection's default region; IAM uses its global endpoint. The code records a regions array as metadata, but collection does not iterate over that array.

Service	Checks performed
IAM	Account password-policy presence; access keys older than 90 days; wildcard principals in role trust documents.
Amazon S3	Default-encryption configuration response; all four Public Access Block flags; public bucket-policy status.
Amazon EC2 / EBS	IPv4 ingress rules containing 0.0.0.0/0; unencrypted EBS volumes. Admin and database ports receive higher severity.
Amazon RDS	PubliclyAccessible and StorageEncrypted configuration flags.
AWS Lambda	Function URLs with AuthType NONE; environment-variable names matching SECRET, TOKEN, PASSWORD or KEY.
AWS CloudTrail	Trail presence, IsLogging state, and root ConsoleLogin events in the returned event sample.
Amazon GuardDuty	Detector presence and imported findings, with numeric GuardDuty severity mapped into application severity.

Evidence interpretation

These checks evaluate specific fields and response patterns. An exposed configuration flag is not an end-to-end network reachability test. A secret-like environment-variable name is a heuristic, not proof that its value contains a credential. Control references describe mappings associated with a check; they do not establish compliance certification.

The S3 encryption check reports the configuration response observed by the collector. Its finding text should be interpreted with that evidence, rather than as proof that every existing object is unencrypted. Likewise, a KMS rotation flag or an ECR scan-on-push flag alone does not describe every policy or scanning mode that may apply.

5 Coverage, findings, and scoring

Additional backend branch	Evaluated signal
KMS	KeyRotationEnabled is false on a successful rotation-status response.
ECR	Repository imageScanningConfiguration.scanOnPush is not enabled.
Secrets Manager	Secret RotationEnabled is not true.
EKS	Cluster endpointPublicAccess is true.
VPC	VPCs exist and the region returns no flow-log identifiers.

Collection scope

The collectors use bounded samples: up to 25 IAM users, 50 role blocks, 30 S3 buckets, 30 Lambda functions, 25 KMS keys, 30 ECR repositories, 30 secrets, and 10 EKS clusters. GuardDuty examines the first detector and contributes up to 20 findings after requesting up to 50 identifiers. The root-login correlation requests up to 10 ConsoleLogin events. These bounds define the reviewed sample, not exhaustive account inventory.

Finding representation

A finding includes its audit and owner, service, check identifier, severity, title, resource ARN, region, evidence, control references, and account identifier. The deduplication key combines account, region, check, and resource. The pipeline initially sets confidence to 0.9; this is a fixed application value rather than a calibrated probability.

Database defaults assign finding scores of 100, 60, 30, 10, and 3 for critical, high, medium, low, and informational severity. Default response deadlines are 24 hours, 7 days, 30 days, 90 days, and 180 days respectively. These are application prioritization defaults.

Account risk score

```
score = min(100, round((25*C + 10*H + 3*M) / max(1, S)))
```

C, H, and M are the counts of critical, high, and medium findings; S is the number of selected service identifiers. A higher score means higher reported risk. Grades are A below 20, B from 20, C from 40, D from 60, and F from 80. The calculation uses the original finding counts, including findings later marked false positive by the critic.

Audit comparison

The pipeline compares deduplication keys with the preceding completed audit on the same connection and records new, absent, and unchanged findings. The separate drift endpoint additionally uses lifecycle status to identify reopened findings. In this context, drift means finding differences between audits; it is not a Terraform-state comparison.

6 Models and investigation tools

Role or endpoint	Configured model and output
Audit critic, attack paths, fix drafting	google/gemini-3-flash-preview via Lovable; structured verdicts, path graphs and snippets.
Impact / permissions / principal suggestions	google/gemini-2.5-flash via Lovable; contextual projections and inferred relationships.
CloudTrail replay	google/gemini-2.5-flash via Lovable; behavioral summary, anomalies and risk.
Plan parsing and change review	google/gemini-2.5-flash via Lovable; extracted changes and ship/warn/block assessments.
Remediation action planning	Gemini 2.5 Flash; retries Gemini 2.5 Pro if the returned action list is empty.
Post-change verification planning	Gemini 2.5 Flash; expected-response checks.
Execution failure refinement	Direct OpenAI gpt-4o when OPENAI_API_KEY exists; otherwise Gemini 2.5 Flash via Lovable.

Attack paths and impact

The attack-path prompt requests up to three chains of two or more findings, with a narrative, graph nodes and edges, severity, and a projected blast-radius summary. ReactFlow renders the stored graph. These are model-generated hypotheses derived from finding metadata; the pipeline does not execute an exploit to validate a path.

The impact endpoint loads the latest completed audit for the user and optional connection. A proposed resource change is evaluated against bounded findings and attack-path context. The response includes impacted principals, services, resources or workflows, confidence, preconditions, rollback suggestions, and evidence references.

Effective permissions

The permissions view asks the model to infer effective actions, possible assumable roles, reachable resources, and toxic combinations from the named principal and audit context. The prompt requires best-effort entries and asks uncertain inferences to be marked in notes. It does not retrieve and resolve all IAM policies, permission boundaries, or organization SCPs. This view is an investigation aid, not an authoritative IAM authorization calculation.

6.1 Principal replay and plan review

CloudTrail principal replay

The replay endpoint verifies connection ownership, extracts the final path segment of the principal ARN, and queries CloudTrail LookupEvents using the Username filter. It operates in the connection's default region over a window clamped to 1–90 days. Pagination is limited to ten pages of fifty events, so the maximum retrieved sample is 500 events.

The backend counts event names, keeps the top 25 APIs, and passes up to 30 recent events to Gemini for behavioral analysis. The model receives event names, timestamps, source IPs and error codes extracted from the event sample. The saved replay contains the selected window, event count, top APIs, raw sample, anomalies, narrative and model-generated risk score.

The displayed risk describes the sampled history. Username filtering and event sampling define its scope; the replay is not a complete reconstruction of every role session or every activity source.

Infrastructure-plan review

The operator pastes plan text and optionally selects a connection for audit context. The endpoint accepts Terraform plans, Terraform JSON output, CloudFormation changes, and general diffs as model-interpreted inputs. It truncates input at 60,000 characters and rejects text shorter than 20 characters.

Gemini first extracts at most 25 proposed changes. The parser prompt prioritizes destructive changes and security-sensitive resources. Each extracted change is then reviewed in parallel against up to 60 findings and eight attack-path summaries from the latest completed audit.

Each review can include a ship, warn, or block verdict; risk and confidence; issues; impacted entities; preconditions; rollback steps; evidence references; and a short copyable PR comment. The overall result takes the strongest verdict, with block above warn above ship. A per-change model-call failure becomes a warning with confidence zero.

Resource selection

The resource-listing endpoint provides live AWS choices for IAM users and roles, S3 buckets, security groups, KMS keys and aliases, RDS instances, Lambda functions, and Secrets Manager secrets. These selectors supply concrete identifiers to the investigation tools. They do not convert the subsequent model assessment into a policy simulation or a tested change.

7 Remediation execution flow

Remediation execution begins with an approved record. Model planning and failure analysis are distinct from the code that signs and sends AWS requests.

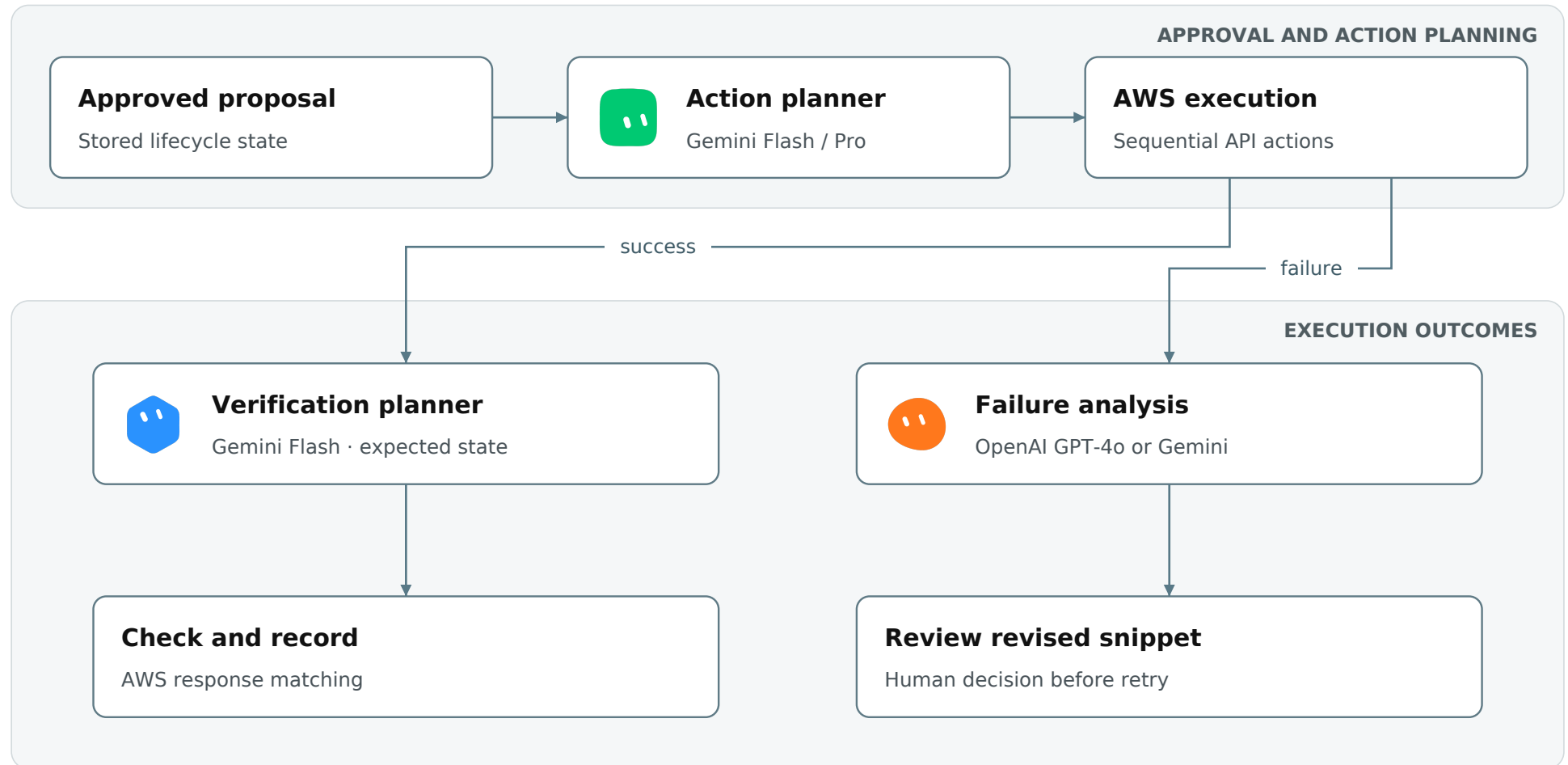


Figure 3. Successful actions proceed to verification; failed actions produce a separately reviewed refinement.

A dry-run request returns a proposed API plan without executing it. Execution requests generate their own plan, stop at the first failed action, and preserve AWS responses. Successful response checks can mark the remediation verified and resolve the finding. Failure refinement does not automatically execute the replacement snippet.

8 Remediation behavior and controls

Proposal and approval

The audit pipeline drafts fixes for up to eight high or critical findings. A remediation stores its snippet, fix type, risk, description, and linked finding. The lifecycle component supports proposed, reviewed, approved, executed and verified states, plus rejection and recorded rollback states. Updates record actor information and timestamps through database triggers.

The apply-remediation endpoint authenticates the user, loads an owned remediation, and requires `lifecycle_state = approved` for execution. The Audit Detail shortcut can set a proposed or reviewed record directly to approved before invoking execution. The connection's separate-approver preference is checked in the lifecycle interface; it is not a universal server-enforced two-person approval rule.

Planning and execution

The planner translates the stored snippet into at most eight structured actions. Supported service adapters cover IAM, S3, EC2, RDS, KMS, CloudWatch Logs, CloudTrail, Lambda, Secrets Manager and GuardDuty. Their protocol handlers construct signed HTTP requests. Terraform, CloudFormation and CLI text are interpreted into API actions; the function does not launch Terraform or a shell.

Actions run sequentially and stop at the first failure. The application stores the action list, API names, response previews, status codes, execution output, and AWS Console links. AWS permissions on the configured credential determine whether a requested change is allowed.

Verification and refinement

After successful execution, Gemini proposes up to four checks intended to read the changed state. The application dispatches them through the same service adapter and tests response success plus case-insensitive contains/not-contains expectations. If every check matches, it records verification and resolves the linked finding. Coverage depends on both the generated checks and adapter support: the S3 adapter handles selected writes, while the Lambda adapter uses a configuration-update route. The lifecycle interface also offers a separately recorded manual verification.

On execution failure, the model receives the snippet, failed action parameters, AWS error and response. It generates an explanation and a revised snippet for review. The original AWS output is stored separately from this model analysis. Recording a rollback state documents an operator action; the provided endpoint does not implement an automatic reverse-action engine.

For production use, approval should be understood as approval of the stored remediation state. A preview is not cryptographically bound to the plan generated later for execution, and prompt instructions are not equivalent to a strict API allowlist.

9 Data model and application state

Table or group	Purpose and relationships
profiles	User-facing profile associated with the authenticated user.
aws_connections	Owned AWS connection, account label, access-key fields, region and verification metadata.
audits	Owned run linked to a connection; scope, status, timestamps, summary and risk score.
findings	Audit-linked check results, resource evidence, critic verdict, controls, lifecycle and response deadline.
attack_paths	Audit-linked narratives, model-produced graph JSON and projected blast radius.
agent_transcripts	Audit-linked phase, agent label, sequence, content and structured execution data.
remediations / remediation_events	Finding-linked fixes and lifecycle records; approval, execution, verification and actor evidence.
control_mappings	Check identifiers mapped to CIS, NIST, SOC 2, PCI and MITRE references.
account_risk_scores / audit_diffs	Historical risk results and comparisons between audits.
principal_replays	Connection-linked event sample and behavioral assessment.

Ownership and access

Application tables use row-level security policies that compare `auth.uid()` with `user_id`. The browser accesses rows through the authenticated Supabase client. User-facing Edge Functions generally authenticate the bearer token, verify ownership of the requested object, and then use a service-role client for related reads and writes. The pipeline is invoked internally by start-audit using the service-role token.

The connection schema stores `access_key_id` and `secret_access_key` as text fields, and the browser inserts these fields when creating a connection. The reviewed code does not implement application-level credential encryption or an STS AssumeRole connection flow. Database-platform encryption, if configured, is a separate deployment property.

Live views and history

Audit Detail loads the audit, transcript, findings, attack paths and related remediations, and subscribes to database changes for updates. Findings and dashboards read stored rows; investigation results from impact and plan review are returned to their screens. Replay results and audit differences are explicitly persisted by their endpoints.

10 Backend interface reference

Endpoints are Supabase Edge Functions under `/functions/v1/`. JSON calls use the authenticated session, except the internal pipeline kickoff. The export handler reads query parameters and returns a file response.

Function	Input and result
verify-aws-connection	connection_id ■ STS identity, account and verification result.
start-audit	connection_id, services[] ■ audit_id; creates a queued audit and initiates the pipeline.
run-agent-pipeline	audit_id, user_id ■ immediate acknowledgement; continues collection, analysis and persistence asynchronously.
list-aws-resources	connection_id, service ■ region and resource items with ARN, label and hint.
simulate-impact	mode plus connection_id and target/principal ■ contextual JSON assessment or principal candidates.
cloudtrail-replay	connection_id, principal_arn, days ■ saved replay, counts, sample and model analysis.
review-plan	plan_text, format, optional connection_id ■ parsed changes, individual reviews and overall verdict.
run-drift-diff	audit_id ■ persisted comparison with the prior completed audit or a first-run baseline.
apply-remediation	remediation_id, optional dry_run ■ plan preview or execution actions, outcomes and verification.
export-audit	audit_id and format query parameters ■ CSV findings or a compact summary PDF.

Export contents

CSV export contains severity, service, check identifier, title, resource ARN, region, lifecycle status, deadline, critic verdict, finding score and account. The built-in PDF is a single-page summary with account context, counts, top findings and attack-path titles; it is not a complete evidence archive.

Execution state

Audit records move through queued, running, completed or failed. The starter and pipeline return before the entire audit is finished; the interface observes persisted progress. Transcripts contain both narrative stage messages and structured API-command descriptions. A completed audit denotes completion of this pipeline, rather than comprehensive proof that every AWS resource was evaluated.

11 Configuration and local development

The frontend uses React 18, TypeScript, Vite 5, Tailwind CSS, shadcn/ui, React Router and Supabase JS. ReactFlow supports attack graphs; Recharts supports visual summaries. Backend functions use Deno TypeScript, Supabase JS imports and custom AWS Signature Version 4 helpers.

Frontend setup

```
git clone https://github.com/ritvikindupuri/Blastline-AI.git
cd Blastline-AI
npm install
```

Configure the variable names consumed by `src/integrations/supabase/client.ts`:

```
VITE_SUPABASE_URL=<Supabase project URL>
VITE_SUPABASE_PUBLISHABLE_KEY=<public client key>
```

```
npm run dev
npm run build
npm run preview
```

Backend configuration

Use a Supabase project with the supplied schema migrations and Edge Functions. Local Supabase development requires its CLI and container runtime. Apply migrations to a local or disposable environment first, and configure function secrets in the target environment before exercising the application.

Variable	Use
SUPABASE_URL	Backend project URL.
SUPABASE_SERVICE_ROLE_KEY	Privileged backend database client and internal pipeline invocation.
SUPABASE_ANON_KEY	User-authentication client; some functions accept SUPABASE_PUBLISHABLE_KEY as a fallback alternative.
LOVABLE_API_KEY	Required by the Gemini analysis and remediation routes.
OPENAI_API_KEY	Optional direct GPT-4o route for execution failure refinement.

Configure AWS credentials through Connections and verify the resulting identity before starting an audit. Collection needs permission for the selected read APIs. Remediation additionally needs the specific write permissions used by the approved action. Browser environment variables must contain only public client configuration, never service-role or model-provider secrets.

Repository scripts

`npm run lint` runs ESLint; `npm run test` runs Vitest; `npm run test:watch` starts watch mode. The checked-in test file is a starter example. This documentation review inspected source behavior and did not validate a deployed Supabase/AWS environment.

12 Implementation references and next steps

The references below identify the implementation used for the descriptions and diagrams. They link to a fixed repository revision so later source changes can be compared without silently changing the reference.

Topic	Implementation
Audit collection and model sequence	run-agent-pipeline/index.ts
Authentication and audit kickoff	start-audit/index.ts verify-aws-connection/index.ts
Impact and principal analysis	simulate-impact/index.ts cloudtrail-replay/index.ts
Plan review and audit differences	review-plan/index.ts run-drift-diff/index.ts
Remediation and approval interface	apply-remediation/index.ts RemediationLifecycle.tsx AuditDetail.tsx
Resources and exports	list-aws-resources/index.ts export-audit/index.ts
Persistence and configuration	supabase/migrations Supabase client package.json

Next steps

Extend the regional collection loop and pagination, and expose the additional backend service checks through the audit selector. Strengthen evidence interpretation so unknown or unauthorized responses remain distinct from confirmed misconfigurations.

For remediation, bind approval to a specific validated action plan, enforce approver separation on the server, and restrict both write actions and verification calls with explicit API rules. Move connection secrets into a dedicated credential mechanism and introduce short-lived role sessions.

Add integration tests for AWS response parsing, per-resource finding linkage, authorization, lifecycle transitions, and verification behavior. Scheduled audit and GitHub PR configuration tables exist in the schema; automated scheduling and webhook execution should be treated as future integration work until their execution paths are provided and tested.

Repository revision: [ad42b680ce26a7d531111abca925edb5db95ac09](#)